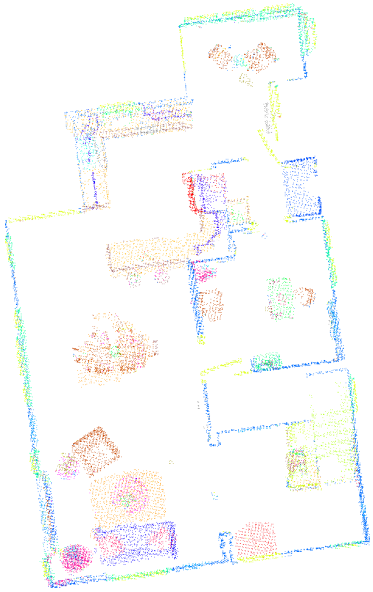
313553007 楊宜祥

Homework2: A Robot Navigation Framework

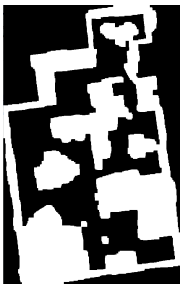
Code Explanation

Construct Semantic Map：get_semantic_map()



在轉換成平面俯視圖之前，我先將牆壁與門框高於45%以上高度的點都先濾除，接著建立一個3D array，將寬設定為600，高則根據點雲的總長寬比而決定：
IMG Height = 600 * PCD Height / PCD width，接著，以PCD中座標位置最小的點作為基準點，計算每個點到該點的比例關係：(PCD - min(PCD)) / (max(PCD) - min(PCD))，在用這個比例關係，去得到該點在影像3D array中的pixel 位置，此scale數值會用於後續2D to 3D時使用，因為每個pixel位置可能對應到多個點，在藉由計算其出現的頻率，來決定該pixel的顏色，此function最後可以得到map image以及對應的2D to 3D scale 及 offset。

Determine Stop points: decide_stop_point()



此部份會用於建立Agent 的Configure Space以及決定最終停止點的位置。藉由剛剛得到semantic map，將其轉成二進制並且用Closing, Opening, Dilation, 等數位影像處理的方式，強化邊緣，並以Contour找到被包圍的黑色小區域，並將其填滿，得到最終不可行走區域（白色）以及可行走區域（黑色），如右圖。

接著用Semantic map取得對應的target的中心位置，再與此Configure map交互使用，找到離target中心最近的黑色Pixel，及視為最終的target Point。

RRT Path Planning: utils.RRT_star.RRT_star

RRT的部份我有實踐，RRT以及RRT*，主要的差別在於有沒有對原本的Node重繞，簡易說明一下流程：

1. 檢查start point 及 goal point 是否佔用到禁止行走區域。

2. Randomly Sample Pixel位置。

Random Sample：For RRT, this is the fully random sample method.

Goal biased sample: 在Goal point 為圓心，畫一個半徑為step size的圓，在範圍內有較高的sample 機率。

Diverse Sample

此處在Sample 有增加一定的機率去採樣，會紀錄哪個Pixel位置有被採樣過，在用Gaussian Filter，以備採樣過得Pixel為中心擴散，接著將這個Filtered Image做Normalize並用1減去整個array，並確保Configure Space中禁止區域的機率為0，最後Normalize使其總和唯一，這樣就得到一個比較少被採樣的區域會有較大的機率被採樣的遮罩。

Centralized Sample

主要是在RRT* 模式下，最後會有一個部份會Refine route，這時會採用centralized sample，一樣使用Gaussian Filter，但會集中採樣在那些已經確認會連接前往Goal Point 的node的周圍，以此來得到更短的路線。

3. Check occupied：這部份會檢查，當前採樣的位置是否重複採樣
4. Nearest & Steer：與論文之方法相同，找到對應的最近點。
5. Check_obstacle_free

這部份主要是藉由，兩個點的位置，用cv2.line畫出一條通兩點形成的線，在用C_space與劃線之交集，判斷是否有撞到牆壁或是使用到禁止行走區域等等。

6. Connect Node

這部份我實踐了一個Tree的結構 Node，每個Node只能對應到一個Parent Node但可以有多個Child Node，藉由這個Node可以紀錄他的位置、id、cost等等。每次決定Parent Node時，都會更新當前Node 的Cost，以供後續RRT* mode時決策使用。

7. Rewire (RRT*)

會以一個新的Node為中心，找出DIST_MAX範圍以內所有的相近的Node，並判斷，若那些點的Parent換成新的Node會不會有較低的Cost，若條件成立則重新設定該Node的Parent Node。

8. Check Reach End

若新的Node的位置在goal point一個Step 範圍內且確認沒有任何的障礙物，則結束Path planning。

9. (Extra) Refinement mode

因為RRT*有找到最少cost路徑的rewire機制，此處特別在加上一個mode，會固定做60次，且每次的採樣都會集中在已經確認連接到goal point的那些node附近的位置，以此來近似optimal solution。

10. Get Route

藉由Tree結構，迭代找出所有的父節點直到父節點為Init Point。此外，如果在Refinement mode之下，會藉由先前的節點、當前的節點、父節點、父節點的父節點，這四個點，在每一次迭代時，藉由一些條件，來減少路徑上所需的節點數，方法如下：

1. 建立temp new node = (current node + parent node) / 2
2. 判斷 prior node to temp new node 和 temp new node to parent of parent node之間有無障礙，若無，則新增新的節點temp new node到路徑，並且直接從parent of parent node開始下一次迭代。

藉由這個方式可以減少route所需的node 數量。

Navigation: navigate()

初始時，會先將藉由上述提及的scale將每個route pixel location轉換成PCD coordinate，設定每個move unit: 0.05，turning angle unit: 1 degree，並且會將朝向設定為第二個點，並且在後續迭代時都會先往前移動後再做轉向的旋轉。

往前移動的次數：
moving forward= Distance(Current Agent Position, i th point location) / moving-forward unit

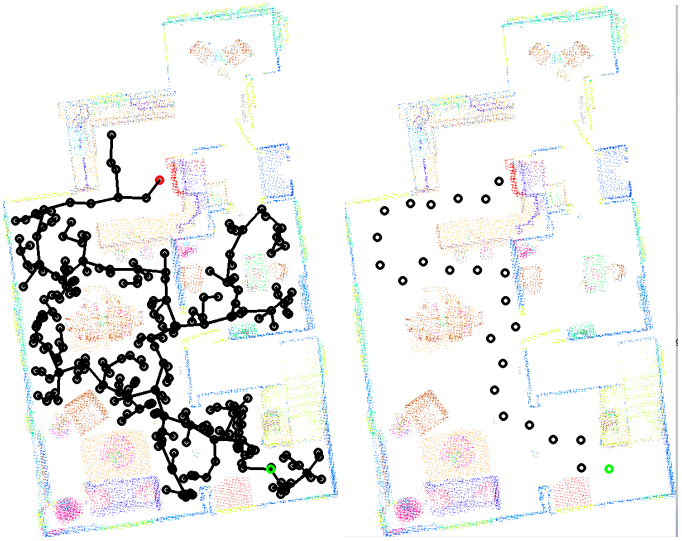
旋轉次數的計算相對複雜，會計算兩個vector
v1 = location after moving forward - location before moving
v2 = Goal point for next iteration - location after moving forward
夾角 theta = arccos (dot(v1, v2) / (norm(v1) * norm(v2))
判斷左轉右轉則是
action = “turn_left” if cross(v1, v2) > 0 else “turn_right”
藉此得到角度，一樣轉動次數：
turning = theta / turning angle unit

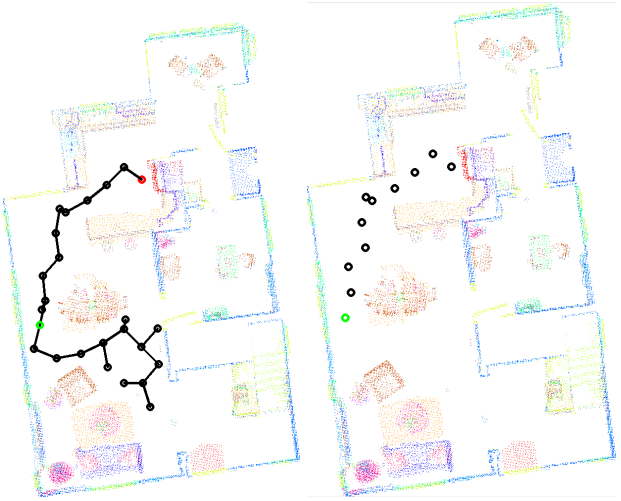
以此，每次都會先移動在旋轉朝向，最後會朝向target的中心位置。
另外，在影片中是使用alpha blending將target顯示成近乎紅色的目標物。

Result & Discussion

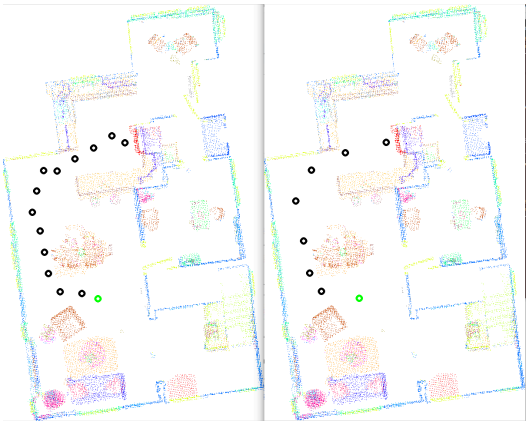
Compare with different start point

此處都是在相同的設定之下，不同的起點產生之結果。左圖為sample 結果，右圖則是最終的路線，可以看到，都是採用隨機採樣的模式，當起點越遠，採樣數量會大幅提高，且也有很多不見得有效的採樣存在在過程之中。



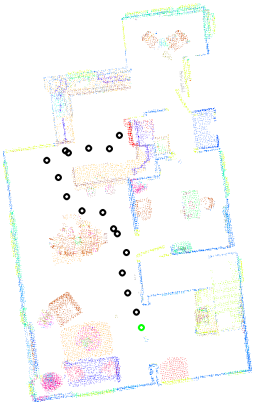


Result for post precessing refined method



此處顯示了有使用Refine以及沒使用Refine的差別，右圖有經過refine，左圖沒有，可以看到，右圖大量的減少了點的數量，且產生的路徑也相對平滑，也顯示了這個方法的有效之處。

Robot Navigation Results



為了評估Navigation 的影響，我設定了兩組參數：
Setting 1: Moving unit : 0.05 , Turning unit: 1
Setting 2: Moving unit : 0.5 ， Turning unit: 5
並且計算，每次迭代時，目標點與移動後的Agent位置的 Mean least square error，左圖為此次實驗採用的路徑點。
就結果而言，
Setting 1 MLSE : 0.00023162507351304846
Setting 2 MLSE : 0.08214100732216392
Setting 2在過程中會有較大的誤差，但因為我採用的方法是根據Agent position 決定下一次的轉動及移動，並不會有Error Propagation的狀況發生。但是仍值得注意的是，有可能因此而碰撞到障礙物，導致路徑的偏差，尤其在當 moving 及 turning unnit的參數設計的越容易發生。

MLSE
N: # route nodes.
Agent_position_i: agent position after moving in i th iteration
Goal_point_i : goal point in i th iteration
MLSE = Summation (Eulcidean Distance(Agent_position_i - Goal_point_i) **2) / N

Reference

RRT_star algorithm:
https://vslam.net/2021/04/10/route_planning/%E8%B7%AF%E5%BE%84%E8%A7%84%E5%88%92%EF%BC%88%E4%B9%9D%EF%BC%89-RRT-Star%E7%AE%97%E6%B3%95/

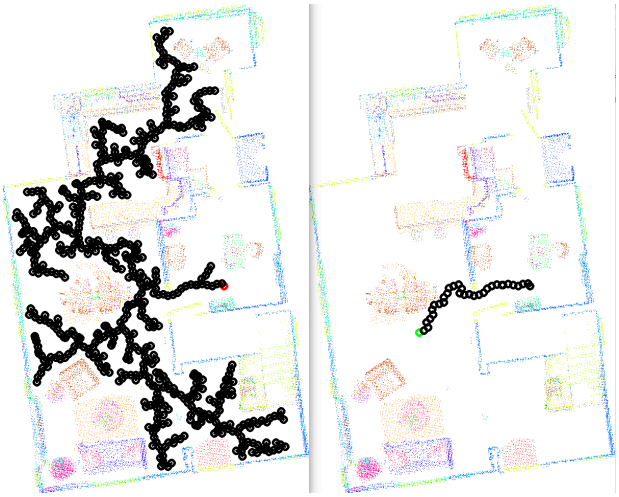
Question

a. For adjust step size:

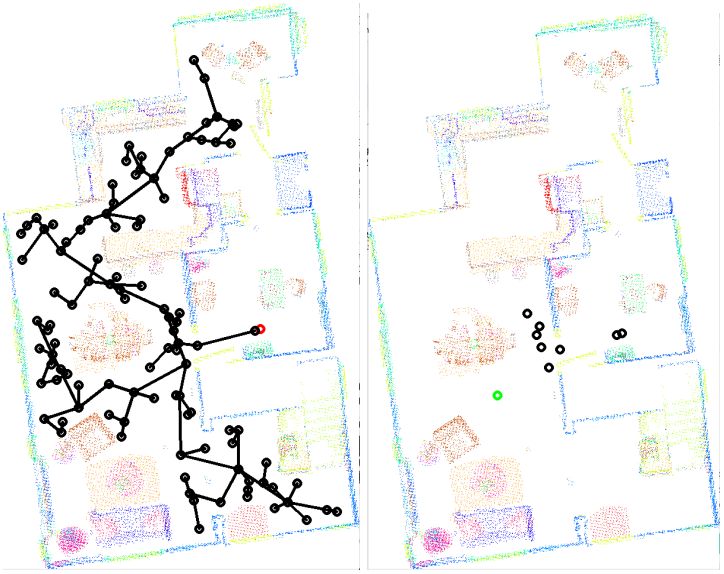
下方式兩組不同的圖，代表RRT在sample過程中的採樣點，紅色圓圈代表Goal Point，綠色代表起點。

可以看到，越小的Step Size，代表每次前進的步伐越小，相對的就會很完整的Survey完Configure Space，且因此，得到的路徑也比較滑順。與large step size相比，large step size大幅減少了運算時間，但得到的結果相對崎嶇，在後續引導Agent移動時也需要較長的距離才能走到目標物。

step = 10; Cost Time: 143.64 s



step = 100
Cost Time: 26.69 s



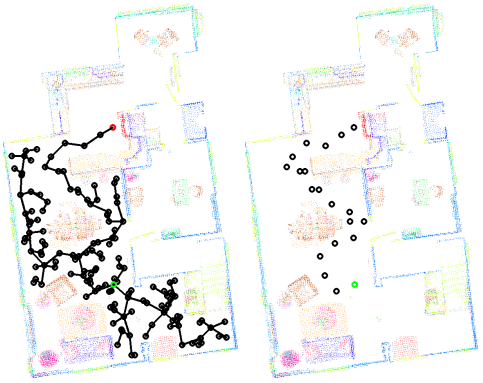
For goal bias:
Goal Bias 會使得，Sample 點接近於Goal point 附近的點機率較高，在相較簡單的環境中，可以較快速的找到Target點的位置，較有目標行的尋找路徑，但若是再有很多障礙物阻擋在前方的地方，一位的朝向Target前進，很有可能因此卡死在角落無法前進；Bias 越小，Agent比較傾向於探索周遭的環境，因為所有的點都是等機率sampled 的，但也有可能Agent一位的探索，而迷失了找到target這像任務目標。。

b. For real world…
若要實踐於Real world，需要考慮路線規劃的動態調整，在不同時間很有可能會有多個物件移動，而導致擋住了原本規劃的前進路線，此時就需要感知，來避開障礙，並也許可能回到原有的規劃路線，又或是重新規劃一條新的移動路線，如何動態的適應環境，同時又更新環境資訊到記憶中，我認為是一個需要被重視且克服的問題點。

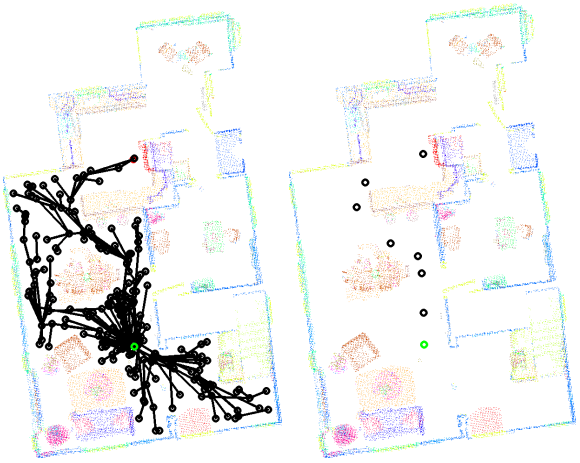
Bonus: RRT*

固定條件：Start point、Stop point、target center

RRT random sample
Cost time: 14.861794233322144



RRT*,random sample, refine
Cost time: 20.924679279327393



正如上述所提及的，RRT* 會Rewire連接過得點，近似最小路徑，其最後散佈點的，有部份特別集中，是因為Refinement的機制，會在這個階段，針對已知路徑灑更多的sample point,

以此來優化路線的cost，整體而言RRT 較為散亂，RRT*較有結構性。

最後在得到最終的路線時可以看到，RRT*的路徑點較少，是因為有採用Refinement post processing來減少路徑點數量，可以看到RRT會有很多轉彎處，而RRT*藉由這個Post processing的手法，會盡量走直線為主。